第三章 栈和队列 · C语言知识准备

> 配套：《第三章 栈和队列的结构分析与应用-教案》《第三章 栈和队列的结构分析与应用-同步练习》
> 定位：栈和队列的算法本身比线性表**简单**（只在两端操作），真正卡住初学者的是三件事——**下标 `top` 的边界**、**取模 `%` 的环形走动**、**指针传址**。这一章就把这三件事一次讲透。

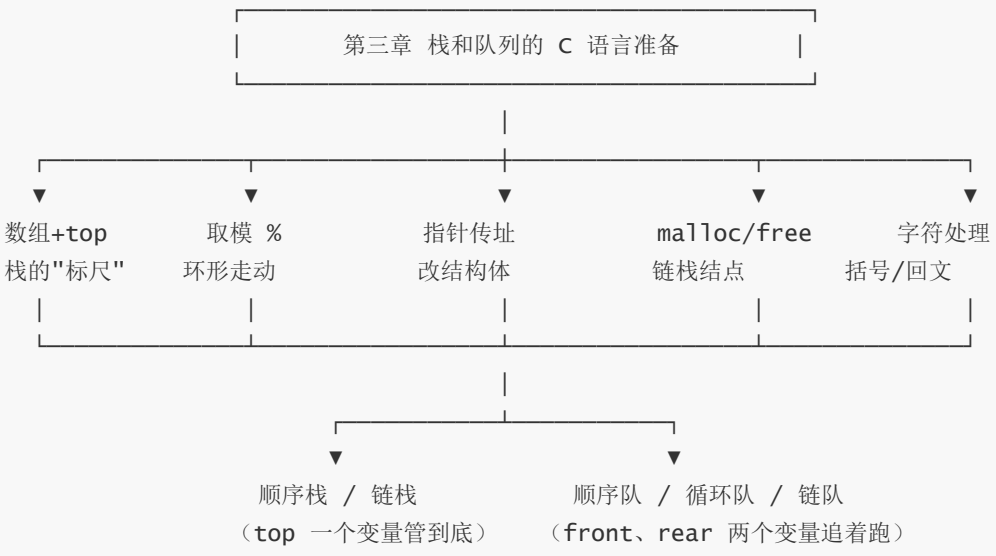
---

0. 先看清"要补哪些 C 知识"

栈和队列本质是**受限的线性表**：栈只在表尾（栈顶）增删，队列只在一端进、另一端出。所以第二章用过的语法大部分**直接复用**，只需再补下面这几块：

栈 / 队列里的概念	需要的 C 语法	出现在哪里
顺序栈（数组 + 栈顶指针）	结构体嵌套、数组下标	`SeqStack { data[StackSize]; top; }`
栈顶位置随时变	整型变量当"指针"用（`top++` / `top--`）	`Push` / `Pop`
循环队列首尾相接	**取模运算** `%`	`(rear+1)%QueueSize`
所有函数都要改栈 / 队列	**指针传址**（`SeqStack *S`）	`InitStack(&S)`
链栈结点	自引用结构体、`malloc` / `free`	`StackNode { data; next; }`
括号匹配、回文判断	字符数组、`'\0'`、`getchar`	栈的应用
出栈元素要"带回来"	指针作输出参数	`Pop(&S, &x)`
判断空 / 满 / 成功	返回值 1 / 0	所有 `StackXXX`、`QueueXXX`

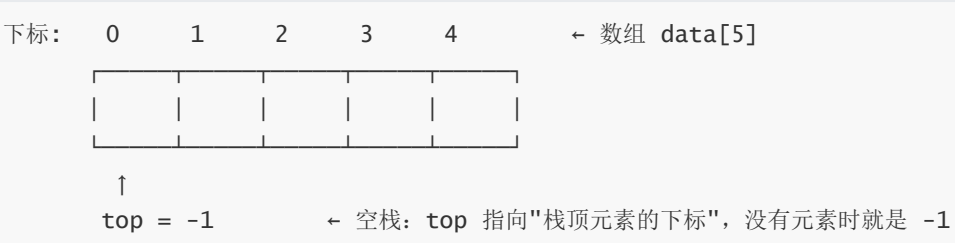
知识地图：



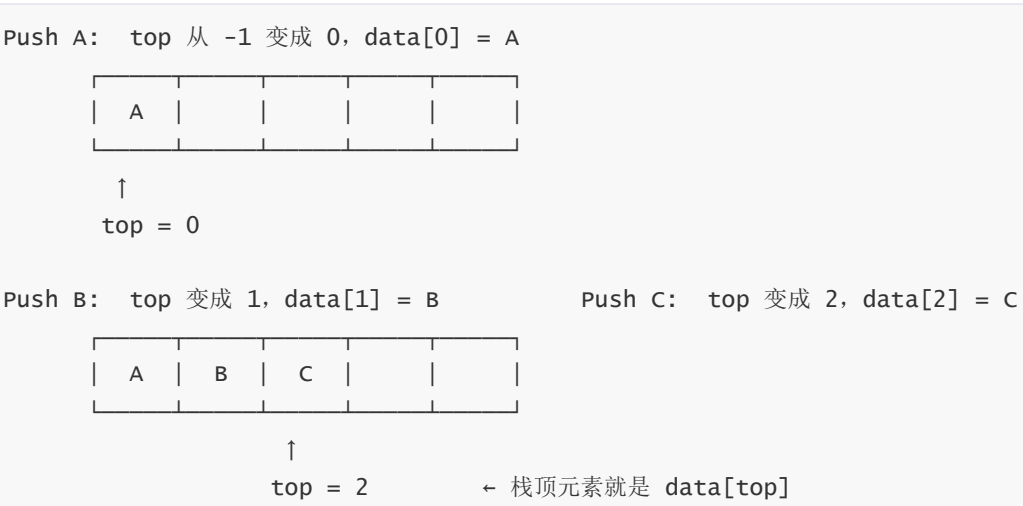
> **一句话预习：** 栈是"一个 `top` 在数组上上上下下"，队列是"`front`、`rear` 两个人在环形跑道上追着跑"。

1. 栈的存储：其实就是一个数组加一个 `top`

1.1 先看"空栈"长什么样



1.2 入栈：先挪 `top`，再放数据



入栈两句话（顺序不能反）：

```
S->top++;                    // ① 栈顶先往上挪一格
S->data[S->top] = x;         // ② 再把新元素放进栈顶
```

也可以合成一句：`S->data[++S->top] = x;`（先 `++` 再取下标）。

1.3 出栈：先取数据，再把 `top` 往下挪

```
x = S->data[S->top];         // ① 先把栈顶元素取出来
S->top--;                    // ② 栈顶再往下退一格（数据还在，但已经"不算数"了）
```

> **易错提醒：** 出栈**不需要**把 `data[top]` 清成 0，`top` 一退，这块位置就作废了，下次入栈会被覆盖。

1.4 `top` 的三种状态必须记牢

状态	判断条件	图示	此时能做什么
空栈	`top == -1`	`top` 在数组左边外面	只能入栈，**出栈就是下溢**
栈满	`top == StackSize - 1`	`top` 指到最后一格	只能出栈，**再入栈就是上溢**

状态	判断条件	图示	此时能做什么
一般	`0 <= top <= StackSize-1`	`top` 指着某个元素	入栈、出栈都行

> **口诀：** 空栈 `-1`，满栈 `StackSize-1`，元素个数 = `top + 1`。

自测一下： `StackSize = 5` 时，`top = 2` 表示栈里有几个元素？栈顶元素是谁？

（答：3 个，`data[0]`、`data[1]`、`data[2]`；栈顶是 `data[2]`。）

---

2. 结构体套结构体：`SeqStack` 是什么

第二章的顺序表是"数组 + 表长 `length`"，顺序栈是"数组 + 栈顶位置 `top`"——**结构完全同构**：

顺序表 `SqList`	**顺序栈** `SeqStack`
`ElemType data[MAXSIZE];`	`DataType data[StackSize];`
`int length;`（有几个元素）	`int top;`（栈顶在哪）
`L->length++`	`S->top++`
空表：`length == 0`	空栈：`top == -1`
满表：`length == MAXSIZE`	满栈：`top == StackSize-1`

```
#define StackSize 50
typedef int DataType;           // 栈里装什么，改这一行就行

typedef struct {
    DataType data[StackSize];   // 仓库：连续空间
    int      top;               // 标尺：栈顶元素的下标
} SeqStack;
```

> **关键理解：** `top` 不是指针，只是一个**下标整数**。它"指"在哪儿，哪儿就是栈顶。

---

3. 每个函数为什么都写 `SeqStack *S`

因为 **入栈、出栈、初始化都要改栈里的东西**（`top` 和 `data`），必须传地址：

```
void InitStack(SeqStack *S) { S->top = -1; }              // 要改 → 传址
int  StackEmpty(SeqStack *S) { return S->top == -1; }     // 只读，但习惯也传址
int  Push(SeqStack *S, DataType x);                       // 要改 → 传址
```

调用时：

```
SeqStack S2;              // 注意：定义的是"结构体变量"，不是指针
InitStack(&S2);           // 所以调用要加 &
Push(&S2, 10);
```

两个必须分清的写法（初学者最常混）：

写法	含义	用错会怎样
`SeqStack S;`	S 是一个结构体变量	用 `S.top`，写 `S->top` 报错
`SeqStack *S;`	S 是一个指针（还没指向任何东西）	必须先 `S = &S2` 或 `malloc`，否则野指针崩溃

> **记忆法：** 变量用 `.`，指针用 `->`；函数参数是 `SeqStack *S`，调用就写 `&S`。

---

4. 取模运算 `%`：循环队列的灵魂（本章最难点）

4.1 先搞懂 `%` 是什么

`%` 是"求余数"：`7 % 5 = 2`，`10 % 5 = 0`，`0 % 5 = 0`。

生活里的取模：钟表。 11 点再过 2 小时是 1 点，不是 13 点——因为 `(11+2) % 12 = 1`。循环队列就是一条"钟表式"的数组：走到末尾自动回到开头。

```
4 格的循环队列（QueueSize = 4），rear 走到 3 之后，加 1 就回到 0：
        0 ———▶ 1
        ▲      |
        |      ▼
        3 ◀——— 2
```

4.2 三个核心公式（背下来）

要做的事	写法	含义
队尾后移一格	`rear = (rear + 1) % QueueSize`	到末尾就"绕回" 0
队头后移一格	`front = (front + 1) % QueueSize`	出队后队头也绕圈
元素个数	`length = (rear - front + QueueSize) % QueueSize`	加 `QueueSize` 防止负数

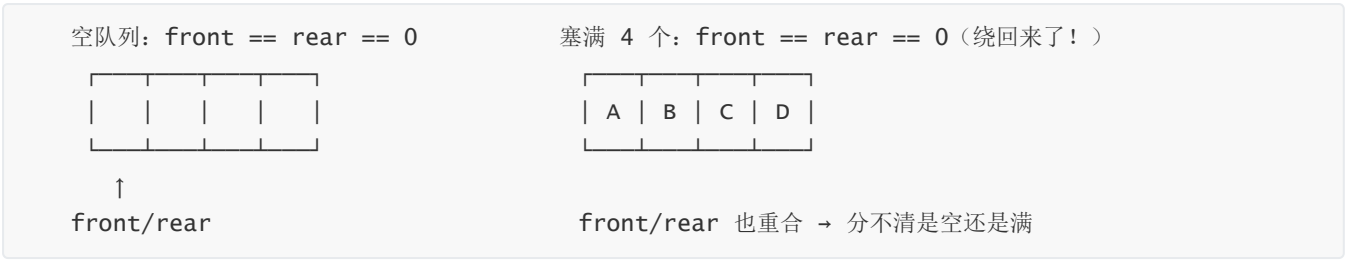
为什么元素个数公式要加 `QueueSize`？ 因为 `rear` 可能绕到 `front` 前面（比如 `rear=1, front=3`），直接减是负数，加上一圈再取模就正常了：

```
rear = 1, front = 3, QueueSize = 4
(1 - 3 + 4) % 4 = 2 % 4 = 2   ✓ 队列里正好 2 个元素
```

4.3 为什么循环队列要"牺牲一个格子"

教材规定：`front` **指向队头元素，** `rear` **指向队尾元素的下一格（下一个空位）。**

这就带来一个尴尬：如果不牺牲格子，`front == rear` 既可能是"空"也可能是"满"，无法区分：



解决办法：故意留一个空位不用。 于是：

状态	判断条件	说明
队空	`front == rear`	两人站在一起
队满	`(rear + 1) % QueueSize == front`	`rear` 再走一格就撞上 `front`
元素个数	`(rear - front + QueueSize) % QueueSize`	最多只能存 `QueueSize-1` 个

> **一句话：** 数组开 4 格，实际最多存 3 个元素，那 1 格是"缓冲区"，用来区分空和满。

4.4 入队 / 出队：谁先动

```
// 入队：先放数据，再挪 rear
Q->data[Q->rear] = x;
Q->rear = (Q->rear + 1) % QueueSize;

// 出队：先取数据，再挪 front
*x = Q->data[Q->front];
Q->front = (Q->front + 1) % QueueSize;
```

> **对比记忆：**
> 栈是**先挪 `top` 再放**（`++S->top`）；
> 队列是**先放再挪 `rear`**（因为 `rear` 指的是空位，本来就该放进去）。
> 两者都遵循同一句话：**"操作完指针要跟着挪"**。

---

5. 链栈：其实就是"只在表头操作的单链表"

第二章学过单链表，`ListInsert(L, 1, e)` 就是插在表头。链栈把这件事固定下来：

- **栈顶 = 链表第一个结点**；
- 入栈 = 头插；
- 出栈 = 删除第一个结点。



```c
typedef struct StackNode {
    DataType data;
    struct StackNode *next;
} StackNode, *LinkStack;

// 入栈（头插）：先接后，再断前
int Push(LinkStack *S, DataType x) {
    StackNode *s = (StackNode *)malloc(sizeof(StackNode));
    if (s == NULL) return 0;          // 申请必判空
    s->data = x;
    s->next = *S;                     // ① 新结点接住原来的栈顶
    *S = s;                           // ② 栈顶指针改指向新结点
    return 1;
}

// 出栈（删首结点）：备份 → 摘链 → free
int Pop(LinkStack *S, DataType *x) {
    StackNode *p = *S;                // p 指向栈顶结点
    if (p == NULL) return 0;          // 空栈，下溢
    *x = p->data;                     // ① 备份数据
    *S = p->next;                     // ② 摘链：栈顶下移
    free(p);                          // ③ 释放
    return 1;
}
```

> **为什么这里用二级指针 `LinkStack *S`？** 因为入栈出栈要改**栈顶指针本身**（让它指向别处），这跟第二章 `InitList(LinkList *L)` 是同一个道理。

链栈 vs 顺序栈：

	顺序栈	链栈
满不满	会**上溢**（`top == StackSize-1`）	只要内存够就不会满
空不空	`top == -1`	`S == NULL`（或 `top == NULL`）
空间	预先开好，可能浪费	用一个申请一个
实现难度	简单，考试常考	需要 `malloc` / `free`

---

6. 字符处理：栈的经典应用都靠它

6.1 字符串还是"以 `'\0'` 结尾的字符数组"

```c
char str[100];
scanf("%s", str);         // 注意：数组名 str 本身是地址，不加 &
int n = strlen(str);      // 需要 #include <string.h>
```

6.2 括号匹配为什么一定要用栈

```
检查 "( [ ] )"：
读到 '(' → 记下来        栈：(
读到 '[' → 记下来        栈：( [
读到 ']' → 和最近一个比对，是 '[' → 配对成功，弹出    栈：(
读到 ')' → 和最近一个比对，是 '(' → 配对成功，弹出    栈：空
全部读完且栈为空 → 匹配正确
```

"最近一个"就是"栈顶" —— 这正是栈"后进先出"的用武之地。

```
// 只处理圆括号的最小版本，先跑通再看括号种类
int Match(char *s) {
    SeqStack st;
    char ch;
    InitStack(&st);
    while (*s != '\0') {          // 顺着字符数组往后走，遇到 '\0' 结束
        if (*s == '(') {
            Push(&st, *s);
        } else if (*s == ')') {
            if (StackEmpty(&st)) return 0;    // 右括号多出来了
            Pop(&st, &ch);
        }
        s++;                          // 别忘了一格一格往后挪
    }
    return StackEmpty(&st);         // 左括号有没有剩
}
```

6.3 进制转换、回文判断为什么也用栈

- **十进制转二进制**：除 2 取余，余数**倒着**写 → 余数进栈，再依次出栈；
- **回文判断**："abcdcba" 正着读、反着读一样 → 全部入栈，再出栈与原串逐个比对。

> 这两个例子说明：**栈不是"为了考试"，而是"天然的逆序工具"。** 只要题目要求"倒过来"，就该想到栈。

---

7. 三个必须背下来的代码骨架

7.1 顺序栈（考试最常考）

```
#define StackSize 50
typedef int DataType;

typedef struct {
    DataType data[StackSize];
    int top;
} SeqStack;

void InitStack(SeqStack *S)          { S->top = -1; }
int  StackEmpty(SeqStack *S)         { return S->top == -1; }
```

```c
int  StackFull(SeqStack *S)          { return S->top == StackSize - 1; }

int Push(SeqStack *S, DataType x) {
    if (StackFull(S)) return 0;      // 上溢
    S->top++;
    S->data[S->top] = x;
    return 1;
}

int Pop(SeqStack *S, DataType *x) {
    if (StackEmpty(S)) return 0;     // 下溢
    *x = S->data[S->top];
    S->top--;
    return 1;
}

int StackTop(SeqStack *S, DataType *x) {   // 取栈顶但不出栈
    if (StackEmpty(S)) return 0;
    *x = S->data[S->top];
    return 1;
}
```

> **教材差异提醒：** 有的教材写成 `DataType Pop(SeqStack *S)`，直接用**返回值**带回栈顶元素。两种写法都行，考试按老师给的函数原型来写，别混用。

7.2 循环队列

```c
#define QueueSize 50
typedef int DataType;

typedef struct {
    DataType data[QueueSize];
    int front;      // 队头元素的下标
    int rear;       // 队尾元素的下一格（空位）
} CirQueue;

void InitQueue(CirQueue *Q)  { Q->front = Q->rear = 0; }
int  QueueEmpty(CirQueue *Q) { return Q->front == Q->rear; }
int  QueueFull(CirQueue *Q)  { return (Q->rear + 1) % QueueSize == Q->front; }

int EnQueue(CirQueue *Q, DataType x) {
    if (QueueFull(Q)) return 0;
    Q->data[Q->rear] = x;                       // 放进空位
    Q->rear = (Q->rear + 1) % QueueSize;        // rear 绕圈后移
    return 1;
}

int DeQueue(CirQueue *Q, DataType *x) {
    if (QueueEmpty(Q)) return 0;
    *x = Q->data[Q->front];
    Q->front = (Q->front + 1) % QueueSize;      // front 绕圈后移
    return 1;
```

```
}

int QueueFront(CirQueue *Q, DataType *x) {        // 取队头但不出队
    if (QueueEmpty(Q)) return 0;
    *x = Q->data[Q->front];
    return 1;
}
```

7.3 链栈

见第 5 节 `Push` / `Pop`，再补一个初始化：

```
void InitStack(LinkStack *S) { *S = NULL; }       // 链栈空：栈顶指针为空
```

---

8. 高频错误对照表

错误写法	后果	正确写法
空栈写成 `top == 0`	少用一个元素，逻辑全错	`top == -1`
`S->data[S->top++] = x;`	先放后挪，放错位置	`S->data[++S->top] = x;`
出栈顺序颠倒：先 `top--` 再取数据	取到的是下一个元素	先 `*x = S->data[S->top];` 再 `top--`
入队写成 `Q->rear = Q->rear + 1;`	走到数组外，越界	`Q->rear = (Q->rear + 1) % QueueSize;`
判断队满用 `rear == QueueSize-1`	循环队列永远判不满	`(rear+1) % QueueSize == front`
元素个数写成 `rear - front`	绕圈后为负	`(rear - front + QueueSize) % QueueSize`
`InitStack(S)`（S 是变量）	传值，改不动外面	`InitStack(&S)`
`S.top` 写在 `SeqStack *S` 的函数里	编译报错	`S->top`
链栈出栈忘了 `free(p)`	内存泄漏	`free(p); p = NULL;`
链栈入栈先写 `*S = s;` 再写 `s->next = *S;`	自己指向自己，链断	先 `s->next = *S;` 再 `*S = s;`
括号匹配里忘了 `s++`	死循环	循环末尾 `s++;`

---

9. 本章小结

知识点	关键记忆
栈	只在栈顶增删，**后进先出**；`top` 是下标，空为 `-1`

知识点	关键记忆
顺序栈	入栈 `++top` 再放，出栈先取再 `--top`，满为 `StackSize-1`
链栈	就是只在表头操作的单链表，入栈头插、出栈删首结点
队列	队尾进、队头出，**先进先出**
循环队列	`(x+1)%QueueSize` 绕圈；空 `front==rear`，满 `(rear+1)%N==front`
取模	钟表思维：走到头就回 0；个数公式要 `+QueueSize` 防负数
指针	改栈 / 队列就传址；改指针本身用二级指针
应用	括号匹配、进制转换、回文、表达式求值——**要逆序就用栈**

---

10. 自测题（先做再对答案）

一、填空

1. 顺序栈空栈的条件是 `top == _____`，栈满的条件是 `top == _____`。
2. 循环队列中元素个数的计算公式是 `_____`。
3. 链栈入栈时，新结点的 `next` 应指向 `_____`。

二、程序阅读

```
SeqStack S;
int x;
InitStack(&S);
Push(&S, 1); Push(&S, 2); Push(&S, 3);
Pop(&S, &x);
Push(&S, 4);
Pop(&S, &x);
printf("%d\n", x);
```

输出是 __，此时栈里从栈底到栈顶的元素是 __。

三、找错

```
int EnQueue(CirQueue *Q, DataType x) {
    if (QueueEmpty(Q)) return 0;
    Q->data[Q->rear] = x;
    Q->rear = Q->rear + 1;
    return 1;
}
```

有 __ 处错误：__。

参考答案

一、填空

1. `-1` ；`StackSize - 1`
2. `(rear - front + QueueSize) % QueueSize`
3. 原来的栈顶结点（`*S`）

二、程序阅读

输出 `4`；栈内从栈底到栈顶是 `1 3 4`。

（过程：入 1、2、3 后栈为 `1 2 3`；`Pop` 得 3；入 4 后为 `1 2 4`；`Pop` 得 4。）

三、找错

2 处：

1. **判空写错了对象**：入队前应判**队满**，`if (QueueEmpty(Q)) return 0;` 改为 `if (QueueFull(Q)) return 0;`；
2. `rear` **移动没取模**：`Q->rear = Q->rear + 1;` 走到数组末尾就越界了，应改为 `Q->rear = (Q->rear + 1) % QueueSize;`。

---

11. 指针与取模专项练习（★ 必做，★★ 提高，★★★ 挑战）

> 用法：先照着敲、运行看结果，再"改一改"，最后答思考。看不懂的可以跳过 ★★★，不影响完成实践部分。

练习 1（★）：`top` 上上下下——最小的栈

任务： 不写函数，只用 8 行代码体会 `top` 的移动。

```
#include <stdio.h>

int main() {
    int data[5];
    int top = -1;               // 空栈
    int i;

    data[++top] = 10;           // 入栈 10
    data[++top] = 20;           // 入栈 20
    data[++top] = 30;           // 入栈 30

    printf("top = %d，栈顶元素 = %d\n", top, data[top]);

    top--;                      // 出栈一次（30 被弹出）
    printf("出栈后 top = %d，栈顶元素 = %d\n", top, data[top]);

    printf("栈内从底到顶：");
    for (i = 0; i <= top; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");
    return 0;
}
```

运行结果：

```
top = 2，栈顶元素 = 30
出栈后 top = 1，栈顶元素 = 20
栈内从底到顶：10 20
```

要点： `top` 只是下标；`data[top]` 永远是栈顶；出栈只改 `top`，数据不清零。

改一改（必做）： 再入栈一个 40，打印 `top` 和元素个数（`top + 1`）。

练习 2（★）：给 `%` 画个钟表

任务： 观察取模怎么会"绕回来"。

```c
#include <stdio.h>

int main() {
    int i;
    for (i = 0; i < 10; i++) {
        printf("%d -> %d\n", i, i % 4);
    }
    return 0;
}
```

运行结果：

```
0 -> 0
1 -> 1
2 -> 2
3 -> 3
4 -> 0
5 -> 1
6 -> 2
7 -> 3
8 -> 0
9 -> 1
```

要点： 结果永远落在 `0 ~ 3`，这就是"环形数组"不会越界的原因。

改一改（必做）： 把 `4` 换成 `QueueSize`（自己 `#define` 成 6），看输出范围变没变。

练习 3（★★）：循环队列走一圈（填空）

任务： 补齐 3 个空，让 4 格队列依次入队 A、B、C，再出队两次。

```c
#include <stdio.h>
#define N 4

int main() {
```

```
    char q[N];
    int front = 0, rear = 0;
    char x;

    q[rear] = 'A';  rear = (rear + 1) % ______;     // ①
    q[rear] = 'B';  rear = (rear + 1) % N;
    q[rear] = 'C';  rear = (rear + 1) % N;

    printf("元素个数 = %d\n", (rear - front + ______) % ______);   // ②

    x = q[front];  front = (front + 1) % N;
    printf("出队：%c\n", x);
    x = q[front];  front = (front + 1) % N;
    printf("出队：%c\n", x);

    printf("剩余个数 = %d\n", (rear - front + N) % N);
    return 0;
}
```

答案： ① `N`　② `N`、`N`

运行结果：

```
元素个数 = 3
出队：A
出队：B
剩余个数 = 1
```

要点： 每次 `rear`、`front` 移动都要取模；个数公式加 `N` 防负数。

改一改（必做）： 再加一次入队 `q[rear] = 'D'`，看看 `rear` 是不是绕回 0，此时 `rear == front`，正好是"队满"（4 格只能存 3 个）。

练习 4（★★）：函数传址——把栈顶元素改掉

任务： 补全 `Push`，让 `main` 里的栈真的从空变满。

```
#include <stdio.h>
#define StackSize 5
typedef int DataType;

typedef struct {
    DataType data[StackSize];
    int top;
} SeqStack;

void InitStack(SeqStack *S) { S->top = -1; }

int StackFull(SeqStack *S)  { return S->top == StackSize - 1; }

int Push(SeqStack *S, DataType x) {
```

```
    if (______) return 0;           // ① 判满
    S->top++;
    S->data[______] = x;            // ② 放进栈顶
    return 1;
}

int main() {
    SeqStack S;
    InitStack(______);              // ③ 传地址
    Push(&S, 7);
    Push(&S, 8);
    printf("top = %d，栈顶 = %d\n", S.top, S.data[S.top]);
    return 0;
}
```

答案： ① `StackFull(S)`　② `S->top`　③ `&S`

运行结果：

```
top = 1，栈顶 = 8
```

要点： `S.top` 是变量用法，`S->top` 是指针用法，别混。

练习 5（★★）：括号匹配（先用一种括号）

任务： 补齐核心判断，让 `(())` 输出"匹配"，`(()` 输出"不匹配"。

```
#include <stdio.h>
#include <string.h>
#define StackSize 50

char stack[StackSize];
int top = -1;                       // 用全局变量，先不讲传参

int main() {
    char s[100];
    int i, ok = 1;

    scanf("%s", s);                 // 输入不含空格的括号串

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == '(') {
            stack[++top] = '(';     // 左括号入栈
        } else if (s[i] == ')') {
            if (top == -1) { ok = 0; break; }   // 右括号多余
            ______;                 // ① 一对配对成功，出栈
        }
    }
    if (top != ______) ok = 0;      // ② 左括号有剩余也不算匹配

    printf("%s\n", ok ? "匹配" : "不匹配");
```

```
    return 0;
}
```

答案： ① `top--`　② `-1`

测试： 输入 `(())` → 匹配；输入 `(()` → 不匹配；输入 `)()` → 不匹配。

思考： 如果要支持 `[]` 和 `{}`，出栈时应该怎么判断？（提示：比较栈顶字符与当前字符是否配对）

练习 6（★★★ 挑战）：链栈入栈（先接后、再断前）

任务： 补两行，把新结点压到链栈顶。

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct StackNode {
    int data;
    struct StackNode *next;
} StackNode;

int main() {
    StackNode *top = NULL;            // 空栈

    StackNode *s = (StackNode *)malloc(sizeof(StackNode));
    if (s == NULL) return 1;
    s->data = 99;

    ______ = ______;                  // ① 先接后：新结点接住原栈顶
    top = s;                          // ② 再断前：栈顶改指向新结点

    printf("栈顶 = %d\n", top->data);

    free(top);                        // 只有一个结点，直接释放
    return 0;
}
```

答案： `s->next = top;`（左空写 `s->next`，右空写 `top`）

要点： 顺序反了就再也找不到原栈顶；`malloc` 一定要判空，用完一定要 `free`。

思考： 如果要压入第二个结点，`top` 该怎么变？（答：新结点 `next` 指向当前 `top`，再让 `top` 指向新结点——和第一步完全一样）